МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»

ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ И
КОМПЬЮТЕРНОЙ ТЕХНИКИ

ЛАБОРАТОРНАЯ РАБОТА №4

по дисциплине

«АРХИТЕКТУРА КОМПЬЮТЕРА»

Выполнил:

Софьин Вячеслав Евгеньевич

Проверил:

Пенской Александр Владимирович

г. Санкт-Петербург, 2026

Содержание

1. Задание . . . 3
2. Алгоритмическия язык . . . 3
   2.1. Выражения . . . 3
   2.2. Абстрактное синтаксическое дерево . . . 4
   2.3. Семантический анализ . . . 6
   2.4. Конвенция названий . . . 7
3. Организация памяти . . . 8
   3.1. Описание архитектуры . . . 8
   3.2. Внешние устройства . . . 9
   3.3. Использования памяти транслятором . . . 10
4. Поток компиляции . . . 11
5. Система команд . . . 12
   5.1. Общая информация . . . 12
   5.2. Инструкции с n операторов . . . 12
   5.3. Инструкции переходов . . . 12
   5.4. Описание операнда . . . 12
   5.5. Режим операнда . . . 13
   5.6. Коды операторов: . . . 13
6. Транслятор . . . 17
7. Управляющее устройство . . . 18
8. Тракт данных . . . 19
9. Векторные операции . . . 22

1. Задание

alg | cisc | harv | hw | tick | binary | trap | port | cstr | prob1 | vector

2. Алгоритмическия язык

2.1. Выражения

```
literal ::= number | string | char | "true" | "false" ;
prefix-operator ::= "-" | "!" | "~" | "++" | "--" ;
binary-operator ::= "+" | "-" | "*" | "/" | "%"
                  | "<<" | ">>"
                  | "&" | "|" | "^"
                  | "&&" | "||"
                  | "==" | "!=" | "<" | "<=" | ">" | ">=" ;
assignment-operator ::= "=" | "+=" | "-=" | "*=" | "/=" | "&=" | "|="
| "^=" ;

value ::= literal
          | identifier
          | "this"
          | "(" expression ")"
          | "new" identifier "(" ")"
          | "new" identifier "[" number "]"
          | "[" [ literal { "," literal } ] "]" ;

box ::= identifier
         | value "." identifier
         | value "[" expression "]"
         | value "[" expression ":" number "]" ;

expression ::= value { postfix-operator }
             | prefix-operator expression
             | expression binary-operator expression
             | box assignment-operator expression ;
```

```
postfix-operator ::= "++" | "--"
                   | "." identifier [ "(" arguments ")" ]
                   | "[" expression "]"
                   | "[" expression ":" number "]" ;

arguments ::= [ expression { "," expression } ] ;
```

2.2. Абстрактное синтаксическое дерево

```
type ::= "int" | "bool" | "char" | "void"
    | identifier
    | type "[" number "]" ;

program ::= { declaration | function | class } ;

declaration ::= type identifier [ "=" expression ] ";" ;

statement ::=
      expression ";"
    | declaration ";"
    | "return" [ expression ] ";"
    | "break" ";"
    | "continue" ";"
    | if-statement
    | while-statement
    | for-statement
    | "{" { statement } "}" ;

argument ::= type identifier ;

function ::= type identifier "(" [ argument { "," argument } ] ")"
"{" { statement } "}" ;

class ::= "class" identifier "{" { declaration | function }
"}" ;

if-statement ::= "if" "(" expression ")" "{" { statement } "}"
```

```
      { "else if" "(" expression ")" "{" { statement } "}" }
      [ "else" "{" { statement } "}" ] ;

while-statement ::= "while" "(" expression ")" "{" { statement }
"}" ;

for-initializer ::= declaration | expression ;

for-statement ::= "for" "(" [ for-initializer ] ";" [ expression ]
";" [ expression ] ")" "{" { statement } "}" ;
```

2.3. Семантический анализ

2.3.1. Типизация

Типизация строгая статическая.

Типы делятся на два вида: примитивы и ссылочные. К примитивам относятся int, bool, char. Ссылочными типами данных являются одномерные массивы и классы.

Int, bool, char и массивы можно задавать литералами.

Массивы имеют константную длину. Это нужно, чтобы статический анализ хоть немного работал. Массивами (и слайсы) длины 4 расширены всем набором арифметических операций. Они выполняются с помощью векторного блока.

2.3.2. Стратегия вычислений

Стратегия вычислений - вызов по значению, аргументы вычисляются до вызова функций или методов слева направо, и копируются на стек. Внутри функции можно читать параметры. Так же можно менять, но изменяться он только в области видимости этой функции. Можно передавать ссылочные типы; содержимое того, на что они указывают можно менять. В методах всегда скрыто от программиста первым передаётся аргумент, соответствующий конкретному экземпляру класса.

2.3.3. Области видимости

Область видимости лексическая. Оформляется через фигурные скобки. Локальные переменные из вложенных областей видимости недоступны и считаются неопределёнными. Переменные из области видимости предка доступны транзитивно.

Вся программа должна находиться в одном файле.

Есть локальные, глобальные переменные. Локальные переменные определяются в областях видимости. Глобальные переменные - вне областей видимости.

Есть поля. Поля определяются внутри класса. Все поля публичные - доступны всем, кто имеет ссылку на объект.

2.4. Конвенция названий

Точкой входа в программу является функция с именем «Main».

Обработчики прерываний записываются как функции с именем «interrupt{}», с заполненым номером вектора

Для взаимодествия с внешними устройствами есть 4 функции «iin», «cin», «iout», «cout» для соответственного чтения или записи чисел или символов.

3. Организация памяти

3.1. Описание архитектуры

Реализуется гарвардская архитектура.

Есть аппаратно-реализованный стек, для сохранения адресов возврата. Считаю, что чтение и запись происходит за 1 такт.

Память программы основана на словах в 32 бита. Поддерживает адресацию по 64 битному адресу. Доступ к ним слова в памяти программы последовательный с возможностью условных и безусловных переходов, восстановления адреса из стека. В этой памяти последовательно хранятся все инструкции и процедуры.

Есть аппаратно-реализованная таблица векторов прерываний. Поддерживает хранение до 8 векторов. Считаю, что доступ к ним мгновенный.

Память данных основана на словах в 256 бит. Есть дополнительная схемотехника позволяющая выборочно читать и изменять четверти (64 бит) этого слова. Считаю, что чтение и запись происходят за 1 такт. Чтение четверти работает также за 1 такт. Запись четверти требует двух тактов (первый для чтения всего слова, второй для изменения нужной четверти и записи). Поддерживает адресацию по 64 битному адресу. Адресация идёт по четвертям, а не по самим словам. В этой памяти хранятся данные по усмотрению программиста.

Программисту доступны 8 регистров данных, 8 адресных регистров, память данных, шина внешних устройств.

Считаю, что вектора прерываний и память программы неизменяемы в ходе выполнения программы.

Доступно 9 вариантов адресации.

- 32 битная константа, есть расширение знака
- регистры данных

- адресные регистры
- косвенное обращение к памяти по значению адресного регистра
- косвенное обращение к памяти по значению адресного регистра с его предектрементацией; для этой операции предусмотрена аппаратная реализация
- косвенное обращение к памяти по значению адресного регистра с его постинкрементацией; для этой операции предусмотрена аппаратная реализация
- косвенное обращение к памяти по значению адресного регистра с константным смещением
- косвенное обращение к памяти по значению адресного регистра с смещением по значения регистра данных (есть поддержка D5, D6, D7)
- косвенное обращение к памяти по 32 битной константе

Константы для соответствующих вариантов адресации лежат в памяти программы.

3.2. Внешние устройства

Внешние устройства и процессор связаны общей шиной. Есть линии данных, приоритета устройства, наличия прерывания и вектора прерывания. Устройства работают кооперативно. Устройства не могут выставлять наличие прерывания и записывать свой вектор, если другое устройство уже выставило наличие прерывания. Если устройство видит, что есть другое устройство с приоритетом выше, то оно должно уступить ему.

Прерывание снимается только после взаимодействия портом данных устройства.

Есть 4 устройства: ввод чисел, ввод символов, вывод числе и вывод символов. Они занимают 6 портов.

1. Чтение ввода чисел
2. Чтение или запись вектора прерываний ввода чисел

3. Чтение ввода символов
4. Чтение или запись вектора прерываний ввода символов
5. Запись в вывод чисел
6. Запись в вывод символов

3.3. Использования памяти транслятором

Транслятор моего языка программирования в начале памяти данных хранит статические данные. С 1000-го адреса начинается куча. Очистка этой области не предусмотрена. В конце памяти сверху-вниз (по направлению убывания адресов) растёт стек.

Литералы неизменяемые. К литералам относятся всё напрямую записанные данные к программе. Все литералы хранятся в статических данных. Загружаются в переменные при необходимости.

На этапе компиляции под глобальные переменные выделяется места в статической памяти. При инициализации программы, эти места заполняются нужными значениями. В процессе выполнения они могут изменяться.

Литералы и глобальные переменные хранятся вперемешку.

На стеке хранятся локальные переменные. Сюда попадают параметры функций и методов, а также вытесненные виртуальные регистры.

Int, char, bool занимают и выравниваются по четверти слова памяти.

Ссылочные типы данных хранят свои данные в куче.

Начало массива выравнивается по слову. Данные хранятся последовательно друг за другом.

Объекты выделяют под себя место необходимое для хранения всех своих полей.

Константы в памяти программы используется нуждами компилятора для записи адресов литералов, глобальных переменных и прочих констант для смещений и начального состояния.

Перед вызовы функций, методов оформлены по Cdecl, а именно перед вызовом параметры кладутся на стек в обратном порядке. При вызове сначала выполняется пролог, сохраняющий регистры (D1, D2, D3, D4, D5, A0, A1, A2) и оформляющий stack frame. В конце вызова состояние «до» восстанавливается.

При прерываниях дополнительно сохраняется и восстанавливается NZCV.

Следующие регистры зарезервированы для

- D0 - возвращение результата
- D6 - восстановление из стека регистра данных
- D7 - восстановление из стека регистра данных
- A3 - восстановление из стека адресного регистра
- A4 - восстановление из стека адресного регистра
- A5 - указатель на кучу
- A6 - указатель на stack frame
- A7 - указатель на стек

4. Поток компиляции

1. Строим абстрактное синтаксическое дерево
2. Избавляемся от синтаксического сахара
3. Проводим семантический анализ
4. Строим высокоуровневое промежуточное представление. Оно оперирует бесконечным количеством виртуальных регистров и построением по синтаксическому дереву блок схемы из высокоуровневых инструкций. Тут выражения рабиваются по «Three-address code».
5. Строим низкоуровневое промежуточное представление. Сначала переводим высокоуровневые инструкции в набор низкоуровневых. Также тут занимаемся аллокацией виртуальных регистров. Реализован алгоритм «Linear Scan Register Allocation».
6. Транслируем в машинный код

Проделана некоторая оптимизация. Если восстановленный регистр не используется для косвенной адресации его можно прочитать напрямую из памяти.

Также локальные переменные тоже часто не используются для косвенной адресации. Поэтому их тоже можно использовать на уровне операнда инструкции в виде косвенной адресации с офсетом.

Вообще вся моя оптимизация свелась к использования mem-to-mem операций, там где это возможно. Это работает, так как работа с такими инструкциями оптимизирована на уровня тактов процессора.

5. Система команд

5.1. Общая информация

Поддерживаются mem-to-mem, reg-to-mem, mem-to-reg, reg-to-reg операции.

5.2. Инструкции с n операторов

- [31:25] бит - код оператора
- [24] бит - выбор размера оператора (0 - байт, 1 - 8 байт)
- $[23 - 8i{:}16 - 8i]$ байт - i оператор

5.3. Инструкции переходов

- [63:55] бит - код оператора
- [54] бит - резерв
- [53:0] байтов - адрес перехода

5.4. Описание операнда

- [7:5] - режим операнда
- [4:2] - основной регистр

- [1:0] - смещение

  Смещение задаётся как

- 00 - следующие 32 бита константа
- 01 - D5
- 10 - D6
- 11 - D7

5.5. Режим операнда

- 0x0 - #* - прямая загрузка
- 0x1 - D* - регистр данных
- 0x2 - A* - адресный регистр
- 0x3 - (A*) - косвенная
- 0x4 - (A*)+ - косвенная с постинкрементом
- 0x5 - -(A*) - косвенная с предекрементом
- 0x6 - (A:O) - косвенная со смещением
- 0x7 - (#*) - косвенная с прямой занрузкой

5.6. Коды операторов:

0x0 - HLT

0x01 - MOV.size from, to

- from := #* | D* | A* | MEMORY
- to = D* | A* | MEMORY

0x02 - CMP.size that, with

- that, with := #* | D* | A* | MEMORY
- устанавливает флаги NZCV (that - with)

0x10 - ADD.size from, to

- from := #* | D* | A* | MEMORY
- to := D* | A* | MEMORY
- устанавливает флаги NZCV

0x11 - ADC.size from, to

- from := #* | D* | A* | MEMORY
- to := D* | A* | MEMORY
- устанавливает флаги NZCV и учитывает Carry

0x12 - SUB.size from, to

0x13 - MUL.size from, to

- from := #* | D* | A* | MEMORY
- to := D* | A* | MEMORY
- устанавливает флаги NZCV

0x14 - DIV.size from, to

0x15 - REM.size from, to

- from := #* | D* | A* | MEMORY
- to := D* | A* | MEMORY
- устанавливает флаги NZCV
- выставляет Carry, если деление на ноль

0x16 - AND.size from, to

0x17 - OR.size from, to

0x18 - XOR.size from, to

0x19 - NOT.size from, to

- from := #* | D* | A* | MEMORY
- to := D* | A* | MEMORY

0x1A - LSL.size count, dest

0x1B - LSR.size count, dest

- count := #* | D* | A* | MEMORY
- dest := #* | D* | A* | MEMORY
- выставляет Carry

0x1C - ASL.size count, dest

- count := #* | D* | A* | MEMORY
- dest := #* | D* | A* | MEMORY

0x1D - ASR.size count, dest

- count := #* | D* | A* | MEMORY
- dest := #* | D* | A* | MEMORY
- работает с сохранием знака

0x20 - JMP label

- PC <= label

0x21 - CALL label

- stack.push(pc + 1)
- PC <= label

0x22 - RET

- PC <= stack.pop()

0x23 - INTRET

- PC <= stack.pop()
- NZCV <= (A7)+

0x30 - BEQ label

0x31 - BNE label

0x32 - BGT label

0x33 - BGE label

0x34 - BLT label

0x35 - BLE label

0x36 - BCS label

0x37 - BCC label

0x38 - BVS label

0x39 - BVC label

- Относительно NZCV решается нужен ли переход по адресу

0x40 - VADD a, b

0x42 - VSUB a, b

0x43 - VMUL a, b

0x44 - VDIV a, b

0x45 - VREM a, b

0x46 - VAND a, b

0x47 - VOR a, b

0x48 - VXOR a, b

- a, b := A*

0x49 - VEND c

- c := A*
- нужен для записи в память результата векторной операции

0x50 - IN port, dest

- dest := D* | A* | MEMORY

0x51 - OUT port, source

- source := #* | D* | A* | MEMORY

0x52 - EI

- Включает прерывания

0x53 - DI

- Выключает прерывания

0x60 - VCMPBEQ a, b

0x61 - VCMPBNE a, b

0x62 - VCMPBGT a, b

0x63 - VCMPBGE a, b

0x64 - VCMPBLT a, b

0x65 - VCMPBLE a, b

0x66 - VCMPBCS a, b

0x67 - VCMPBCC a, b

0x68 - VCMPBVS a, b

0x69 - VCMPBVC a, b

- a, b = {A*}
- выводит маску из словам, заполненных 1, в зависимости от условия

6. Транслятор

7. Управляющее устройство

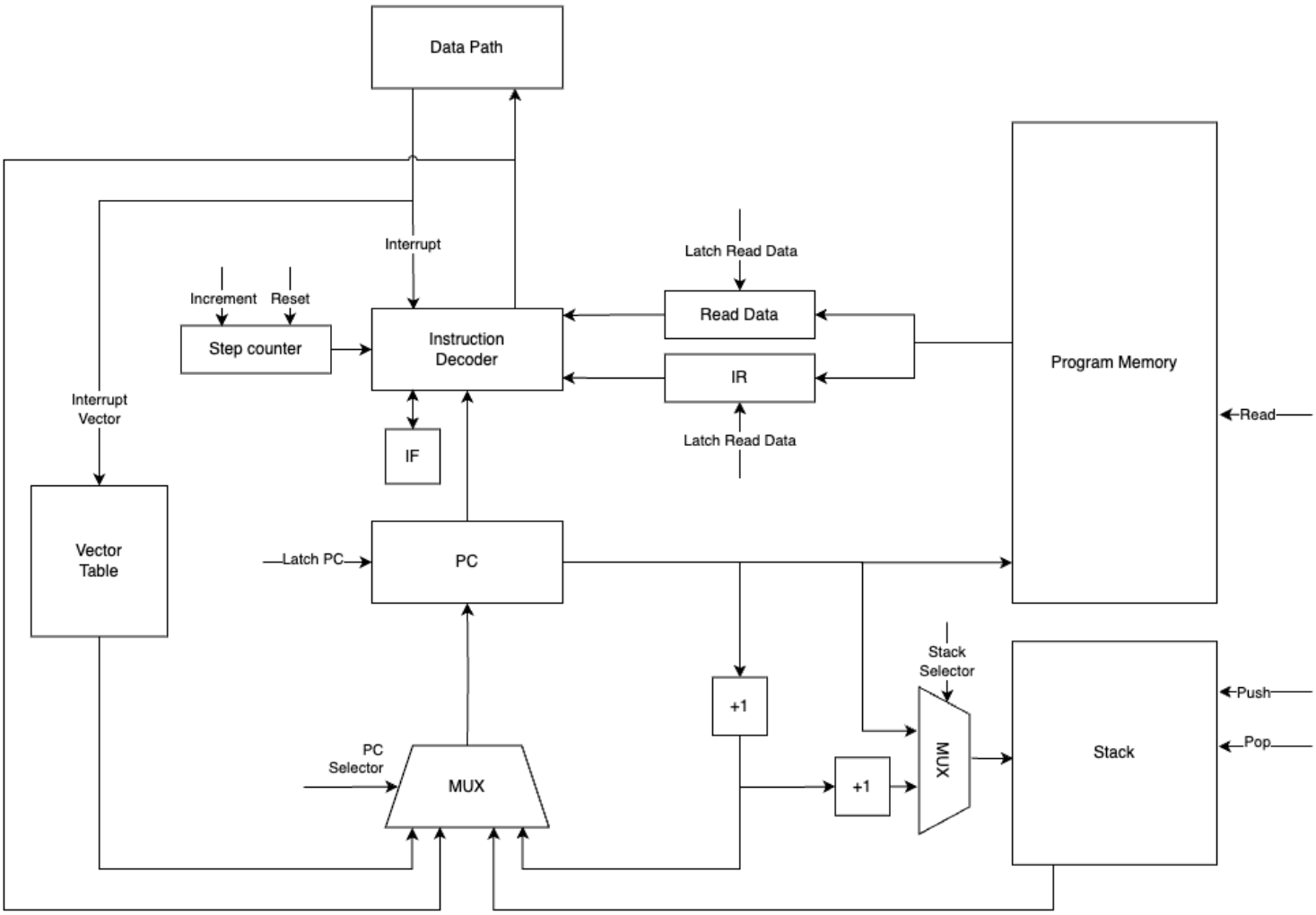
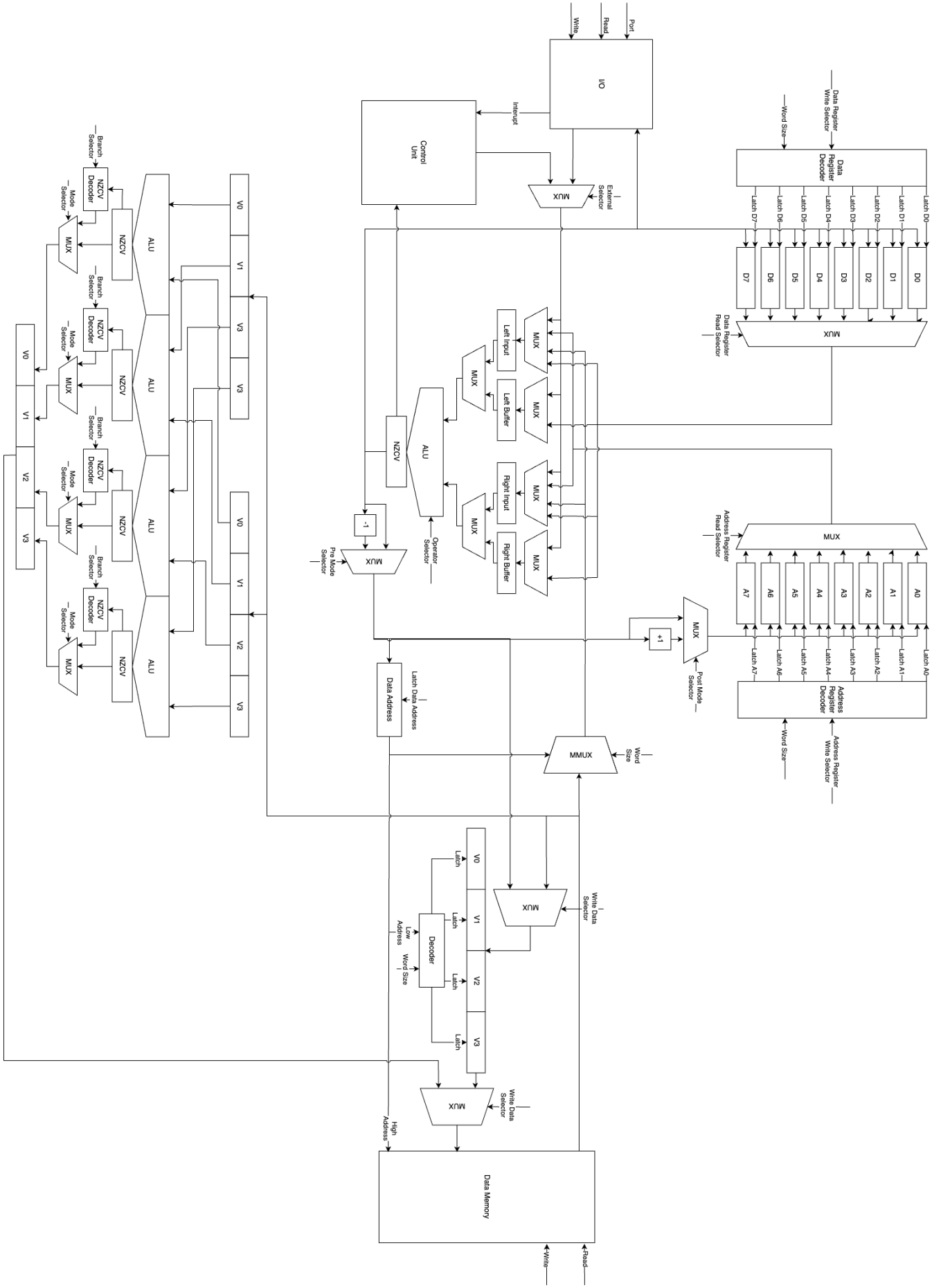


Рис. 1. Схема управляющего устройства

8. Тракт данных



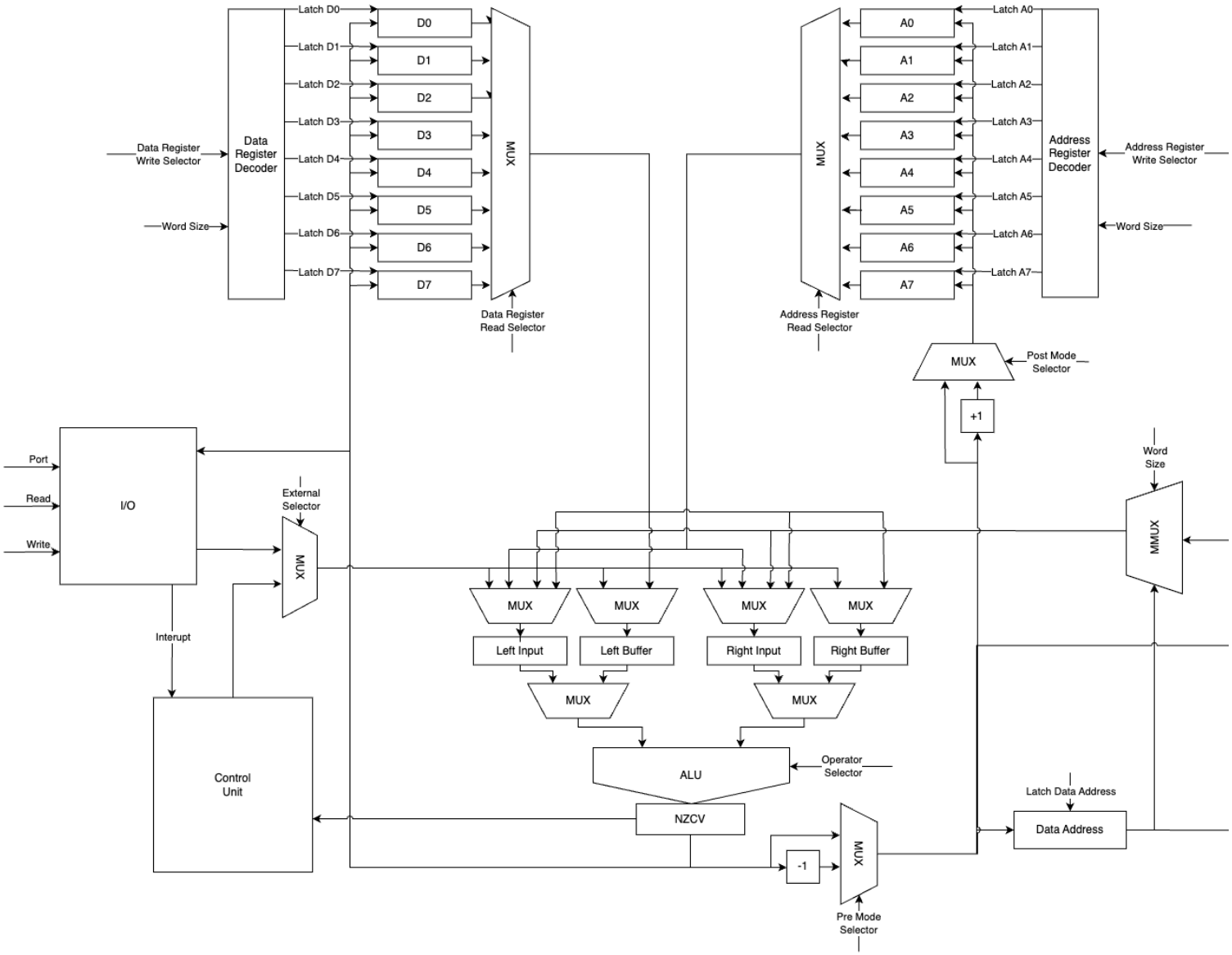


Рис. 3. Схема АЛУ

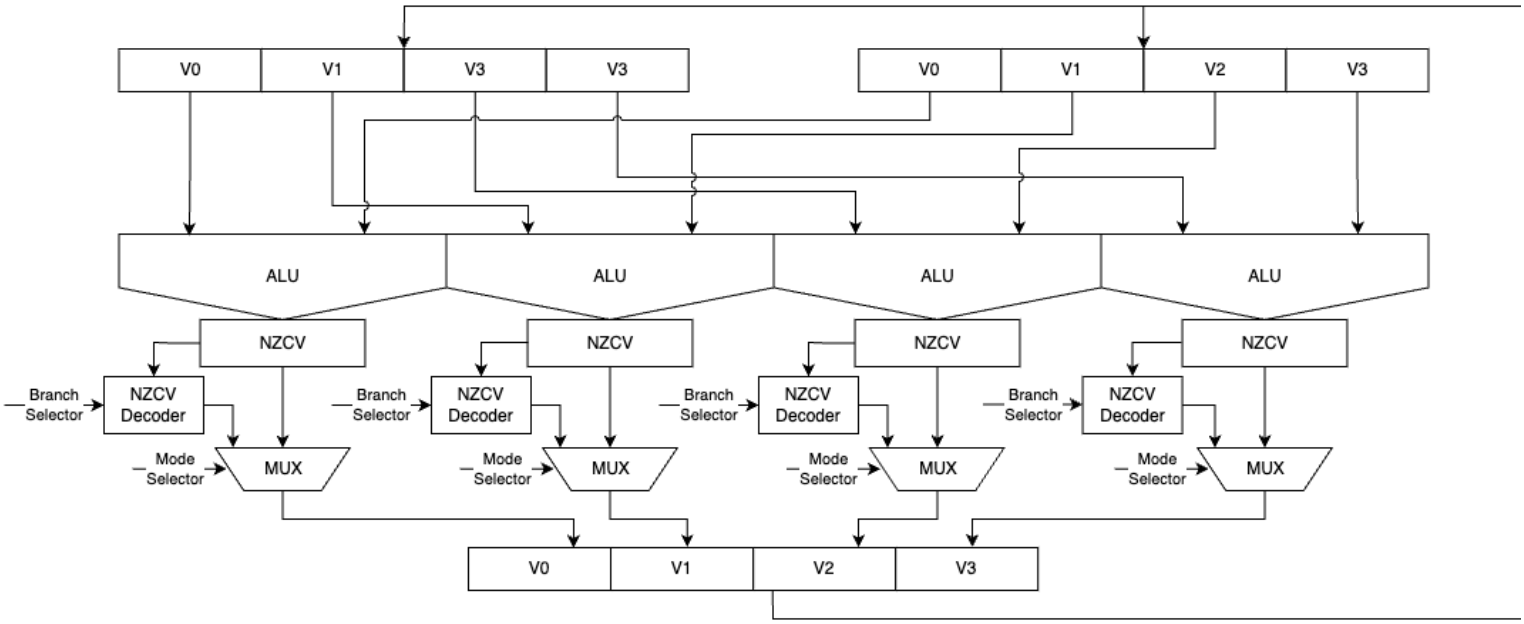


Рис. 4. Схема векторного АЛУ

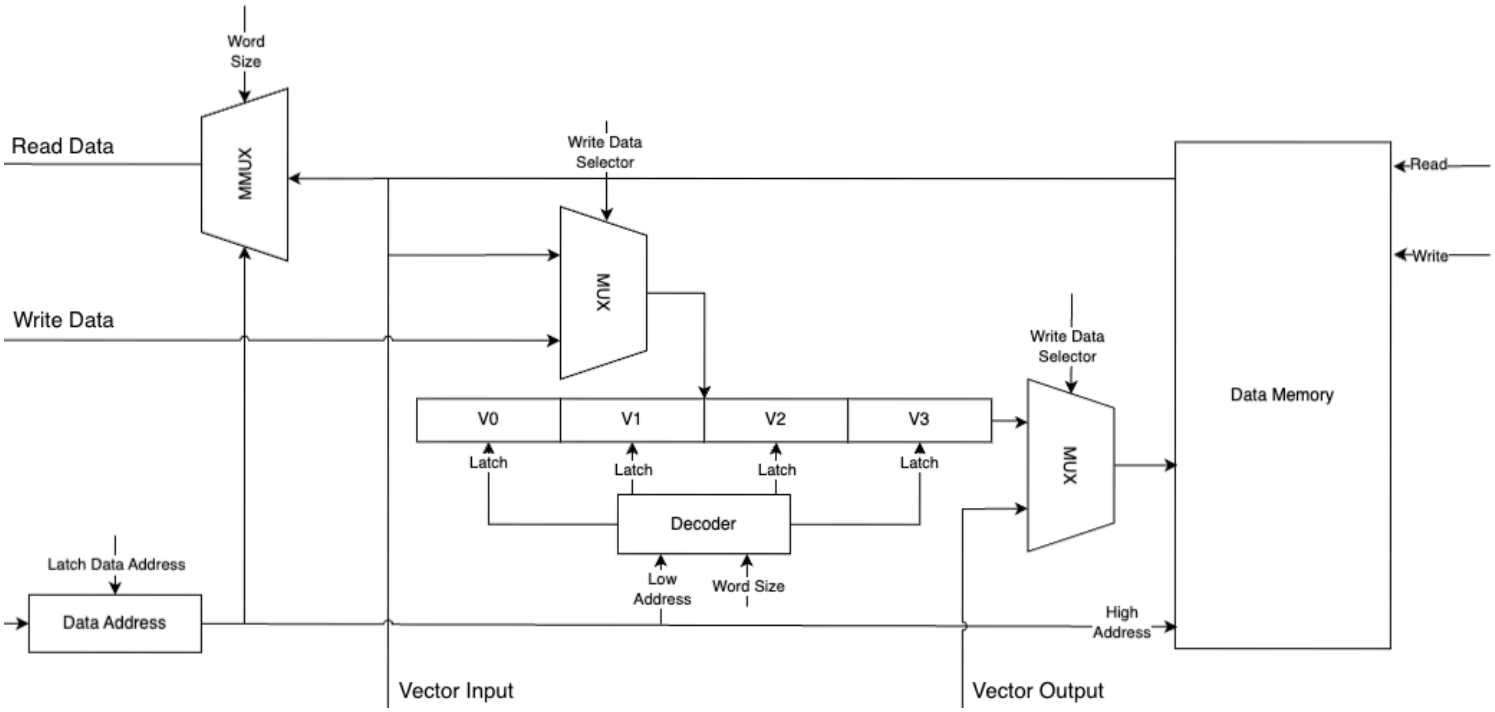


Рис. 5. Схема памяти

9. Векторные операции

	Init	Standard	SIMD
Такты	19859	28185	22010
Норма	0	8326	2151
Скорость	-	100%	387%